

UTS

PENGOLAHAN CITRA



NAMA : Tashfia Wahdah Fuady

NIM : 202431013

KELAS : E

DOSEN : Rizqia Cahyaningtyas, ST, M.Kom

NO.PC : 08

ASISTEN : 1. Joshua Indra Pratama

2. Davina Najwa Ermawan

3. Izzat Islami Kagapi

4. Sakura Amastasya Salsabila Setiyanto

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I.....	3
PENDAHULUAN.....	3
1.1 Rumusan Masalah.....	3
1.2 Tujuan Masalah	3
1.3 Manfaat Masalah	3
BAB II	4
LANDASAN TEORI.....	4
2.1 Pengolahan Citra Digital.....	4
2.2 Konversi RGB ke Grayscale	4
2.3 Operasi Pixel (Brightness dan Contrast)	5
2.4 Histogram Citra	6
2.5 Thresholding dan Segmentasi Warna	8
BAB III.....	9
HASIL	9
3.1 Ekstraksi Kanal Warna, Ambang Batas dan Analisis Histogram	9
3.2 Operasi Pixel.....	16
BAB IV	24
PENUTUP.....	24
DAFTAR PUSTAKA.....	25

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana cara memisahkan channel warna RGB pada gambar digital?
2. Bagaimana cara menganalisis distribusi intensitas warna menggunakan histogram?
3. Bagaimana penerapan thresholding untuk mendeteksi objek berdasarkan warna tertentu pada gambar?
4. Bagaimana cara memperbaiki gambar yang terkena efek backlight menggunakan brightness dan contrast?

1.2 Tujuan Masalah

1. Memahami proses pemisahan channel warna RGB pada gambar digital.
2. Mengetahui cara membaca dan menganalisis histogram warna pada citra digital.
3. Menerapkan teknik thresholding untuk mendeteksi objek berdasarkan warna tertentu.
4. Menghasilkan gambar yang lebih jelas dengan meningkatkan brightness dan contrast pada gambar backlight.

1.3 Manfaat Masalah

1. Menambah pemahaman tentang dasar pengolahan citra digital dan channel warna RGB.
2. Membantu memahami distribusi intensitas warna melalui histogram gambar.
3. Mengetahui cara mendeteksi objek berdasarkan warna menggunakan thresholding.
4. Meningkatkan kemampuan memperbaiki kualitas gambar yang gelap akibat efek backlight.

BAB II

LANDASAN TEORI

2.1 Pengolahan Citra Digital

Citra digital merupakan representasi visual dari objek atau scene dalam bentuk numerik yang dapat diproses oleh komputer. Citra digital terdiri dari kumpulan piksel yang masing-masing memiliki nilai intensitas tertentu. Lebih lanjut, citra digital juga dapat didefinisikan sebagai representasi numerik dari suatu citra dua dimensi (2D). Sebuah citra digital dapat berupa jenis vektor atau bitmap yang bergantung pada resolusi citra tersebut. Citra bitmap memiliki jumlah piksel yang terbatas, di mana banyaknya piksel pada baris dan kolom tidak dapat berubah. Piksel merupakan elemen individu terkecil dalam sebuah citra yang menyimpan nilai keterangan warna pada titik tertentu.

Citra sebagai komponen multimedia mempunyai peran penting sebagai bentuk informasi visual. Dibandingkan dengan teks, informasi yang dimiliki citra lebih kaya dan kompleks. Meskipun demikian, citra seringkali mengalami penurunan kualitas (degradasi) karena noise, warna terlalu kontras, blurring, kurang tajam, dan sebagainya. Oleh karena itu, diperlukan teknik pengolahan citra digital untuk meningkatkan kualitas citra agar lebih mudah diinterpretasikan.

Pengolahan citra (image processing) merupakan suatu sistem di mana proses dilakukan dengan masukan berupa citra (image) dan hasilnya juga berupa citra (image). Peningkatan mutu citra adalah suatu proses mendapatkan citra yang lebih mudah diinterpretasikan oleh mata manusia. Proses-proses yang termasuk dalam kategori peningkatan mutu citra terdiri dari proses-proses yang bertujuan memperbaiki mutu citra untuk memperoleh keindahan gambar, untuk kepentingan analisis citra, serta untuk mengoreksi citra.

Secara umum, pengolahan citra mencakup beberapa proses, antara lain:

1. Image enhancement, yaitu proses perbaikan citra dengan meningkatkan kualitas citra baik kontras maupun kecerahan.
2. Image restoration, yaitu proses memperbaiki model citra.
3. Color image processing, yaitu suatu proses yang melibatkan citra berwarna, baik berupa image enhancement, image restoration, maupun teknik-teknik lainnya.

Pengolahan citra digital memiliki banyak aplikasi dalam berbagai bidang, seperti pengenalan pola, analisis citra medis, pengolahan video, pemantauan lalu lintas, penerbitan, serta bahan pengajaran dan penelitian.

Seiring dengan perkembangannya teknologi, resolusi piksel pada citra digital semakin tinggi sehingga semakin banyak piksel yang perlu diolah. Hal ini menjadikan efisiensi komputasi sebagai faktor penting dalam pengolahan citra digital modern. Penggunaan citra digital semakin meningkat karena kemudahan dalam mendapatkan gambar, memperbanyak gambar, serta kemudahan dalam pengolahan gambar.

2.2 Konversi RGB ke Grayscale

RGB (Red, Green, Blue) adalah salah satu ruang warna yang umum digunakan untuk merepresentasikan citra digital. Setiap piksel dalam citra RGB memiliki tiga komponen: nilai merah (R), nilai hijau (G), dan nilai biru (B). Kombinasi nilai R, G, dan B dapat menghasilkan berbagai macam warna. Citra berwarna umumnya direpresentasikan dalam model RGB yang

terdiri dari tiga lapisan matriks yaitu lapisan Red (R), Green (G), dan Blue (B), sehingga setiap proses perhitungan dilakukan tiga kali pada ketiga lapisan tersebut.

Citra grayscale adalah citra digital yang hanya memiliki satu komponen warna, yaitu intensitas. Nilai intensitas pada citra grayscale biasanya berkisar antara 0 (hitam) hingga 255 (putih). Pemakaian image grayscale disebabkan karena hanya memerlukan sedikit informasi yang diberikan pada setiap piksel dibandingkan dengan citra berwarna, serta ukuran file yang lebih kecil dan proses komputasi yang lebih efisien. Hal ini karena citra grayscale hanya memerlukan satu nilai untuk setiap piksel, berbeda dengan citra RGB yang membutuhkan tiga nilai (R, G, dan B) untuk setiap piksel.

Konversi citra RGB ke grayscale dapat dilakukan dengan berbagai metode, salah satunya adalah dengan menghitung rata-rata dari ketiga komponen warna (R, G, dan B) atau menggunakan formula tertentu yang memberikan bobot berbeda untuk masing-masing komponen. Penggunaan formula yang tepat dapat menghasilkan citra grayscale yang sesuai dengan persepsi visual manusia. Salah satu formula yang umum digunakan berdasarkan standar luminance adalah:

$$\text{Grayscale} = 0.299 \times R + 0.587 \times G + 0.114 \times B \dots(2.1)$$

Formula di atas memberikan bobot yang berbeda untuk setiap komponen warna, sesuai dengan sensitivitas mata manusia terhadap warna-warna tersebut. Terdapat juga variasi formula lain yang sering digunakan, yaitu:

$$\text{Grayscale} = 0.3 \times R + 0.59 \times G + 0.11 \times B \dots(2.2)$$

Kedua formula tersebut pada dasarnya mengikuti prinsip yang sama, yaitu memberikan bobot terbesar pada komponen hijau (G) karena mata manusia paling sensitif terhadap warna hijau, diikuti dengan warna merah (R), dan yang paling kecil adalah warna biru (B). Intensitas citra skala abu-abu tersimpan pada 8 bit integer yang memberikan 256 kemungkinan nilai, dimulai dari tingkat 0 hingga 255, di mana nilai 0 mewakili warna hitam, nilai 255 mewakili warna putih, serta nilai-nilai di antaranya mewakili derajat keabuan.

Dalam implementasinya, konversi citra RGB ke grayscale dapat dilakukan dengan memanfaatkan library OpenCV (Open Source Computer Vision Library), yaitu library open-source yang banyak digunakan dalam pengolahan citra digital dan computer vision. Fungsi `cv2.cvtColor()` pada OpenCV dapat digunakan untuk mengkonversi citra RGB menjadi citra grayscale dengan kode konversi `cv2.COLOR_BGR2GRAY`, yang secara otomatis menerapkan formula konversi berbobot sesuai standar.

Menurut prinsip colorimetry, rentang skala abu-abu dari gambar asli $f(x, y)$ dibagi menjadi beberapa daerah dan diubah menjadi tiga warna primer $IR(x, y)$, $IG(x, y)$, dan $IB(x, y)$ setelah melalui tiga transformasi yang berbeda, yaitu TR, TG, dan TB. Keluaran ketiga matriks ini kemudian digunakan untuk mengontrol saluran merah, hijau, dan biru pada tampilan warna sehingga menghasilkan citra warna pada layar perangkat tampilan.

2.3 Operasi Pixel (Brightness dan Contrast)

Tampilan gambar warna bergantung pada tiga elemen dasar, yaitu kecerahan (brightness), kontras (contrast), dan warna. Peningkatan kualitas citra terutama ditujukan untuk membuat gambar yang telah diproses lebih cocok dalam sistem pengenalan visual sehingga dapat meningkatkan reorganisasi gambar. Tidak semua citra digital memiliki tampilan visual yang memuaskan mata manusia. Ketidakpuasan tersebut dapat timbul karena adanya noise, kualitas pencahayaan pada citra digital yang terlalu gelap atau terlalu terang.

Metode kecerahan citra merupakan salah satu teknik peningkatan mutu citra yang paling mendasar. Proses diawali dengan melakukan penangkapan citra, diteruskan dengan melakukan proses-proses penambahan intensitas kecerahan, penyesuaian kontras citra, maupun penajaman (sharpening). Hasil dari pengolahan citra ditunjukkan dengan adanya perubahan citra yang dihasilkan dan perubahan histogram citra. Pada hasil proses metode kecerahan, citra yang dihasilkan lebih terang dibandingkan citra asli, dan efek pencahayaan yang berbeda dapat dikurangi sehingga warna asli dari sebuah benda dapat terlihat dengan lebih jelas.

Untuk meminimalisir noise (noise reduction) yang disebabkan karena kekurangan kualitas pencahayaan dapat dilakukan dengan menggunakan metode Low-Pass Filter (LPF). Low-Pass Filter adalah proses penghalusan transisi tajam pada sebuah citra yang mengambil data pada frekuensi rendah dan melemahkan frekuensi tinggi. Metode ini mampu menghaluskan transisi tajam pada citra digital serta mengurangi noise yang ada, meskipun efek samping yang dapat timbul adalah citra terlihat sedikit lebih buram (blurry). Persamaan matematika dari metode Low-Pass Filter dapat dinotasikan sebagai berikut:

$$D(i,j) = (1/k) \times W_{3 \times 3} \oplus C(i,j) \dots (2.3)$$

Dengan $C(i,j)$ adalah matriks citra masukan, i dan j adalah indeks baris dan kolom pada matriks, N dan M adalah jumlah baris dan kolom pada matriks, $W_{3 \times 3}$ adalah weighted matrix dari Low-Pass Filter dengan nilai $\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$, k adalah faktor skalar dari $W_{3 \times 3}$ (bernilai 16), dan $D(i,j)$ adalah matriks citra keluaran (hasil).

Transformasi matematika pada operasi piksel juga mencakup operasi terbalik dari citra skala abu-abu, yaitu mengubah warna gelap menjadi terang dan warna terang menjadi gelap (inverse operation). Selain itu, transformasi dengan penambahan konstanta pada seluruh piksel menghasilkan efek peningkatan kecerahan, meskipun dapat menyebabkan tingkat kecerahan citra skala abu-abu semakin berkurang apabila nilai konstantanya melebihi batas ambang tertentu.

Metode ketajaman (sharpening) juga sangat membantu dalam mengolah citra untuk menghaluskan piksel gambar. Tingkat ketajaman ditentukan oleh nilai persentase yang diubah ke dalam bilangan yang berpengaruh terhadap nilai RGB pada setiap piksel citra. Citra yang memiliki kualitas buruk, terutama gambar dengan kontras rendah, dapat memberikan data yang tidak memadai untuk interpretasi visual. Algoritma yang ditingkatkan mampu menghasilkan peningkatan citra yang lebih baik pada sebagian besar citra gelap dengan adanya peningkatan kecerahan rata-rata, warna alami, dan informasi yang lebih rinci.

2.4 Histogram Citra

Histogram gambar adalah bentuk representasi grafis dari distribusi nada atau intensitas dalam sebuah gambar digital. Representasi grafis tersebut merupakan skema dua dimensi dengan nilai integer tingkat abu-abu pada sumbu x dan jumlah terjadinya piksel pada sumbu y . Histogram citra merupakan distribusi probabilitas berupa nilai kecerahan warna yang diukur dari nilai 0 untuk warna gelap hingga 255 untuk warna putih.

Gambar skala abu-abu yang khas diidentifikasi oleh 8 bit per piksel dan histogramnya mencakup 256 tingkat abu-abu. Sumbu horizontal dari plot histogram yang melambangkan tingkat abu-abu yang berbeda dipartisi menjadi sejumlah celah yang diidentifikasi sebagai bins. Histogram gambar digital dengan tingkat abu-abu dalam kisaran $[0, L-1]$ merupakan fungsi diskrit yang dinyatakan sebagai:

$$H(r_k) = n_k \dots (2.4)$$

Di mana r_k adalah tingkat abu-abu ke- k dan n_k adalah jumlah piksel pada gambar yang mempunyai tingkat abu-abu r_k . Untuk menormalkan histogram, setiap nilainya dibagi dengan jumlah total piksel dalam gambar. Histogram memberikan gambaran yang sempurna tentang fitur kontras dan kecerahan dari sebuah gambar.

Histogram mempunyai banyak manfaat pada pengolahan citra, di antaranya untuk menentukan parameter digitasi dan pemilihan batas ambang. Puncak histogram menunjukkan intensitas piksel yang menonjol, sedangkan lebar dari puncak menunjukkan rentang dari kontras gambar. Citra yang memiliki kontras lebih terang atau terlalu gelap memiliki histogram yang sempit, yaitu hanya menggunakan setengah dari daerah derajat keabuan. Citra yang baik memiliki histogram yang mengisi daerah derajat keabuan secara penuh dengan distribusi yang merata pada setiap intensitas piksel. Histogram citra dikatakan baik bila mampu melibatkan semua level atau aras yang mungkin pada level keabuan, agar mampu menampilkan detil pada citra sehingga mudah untuk diamati.

Ekualisasi histogram atau Histogram Equalization (HE) adalah proses merubah nilai dalam sumbu horizontal dan vertikal untuk mendapatkan bentuk keluaran yang berbeda dari aslinya, yaitu proses meratakan seluruh nilai keabuan citra ke dalam warna cerah. Metode HE bisa meningkatkan kontras citra dengan menggunakan pendeteksian nilai piksel minimal dan maksimal dari histogram citra, menyesuaikan kontras citra dengan merata, sehingga area kontras yang rendah menjadi lebih tinggi dan perbaikan kontras akan lebih efektif.

Pemerataan histogram (histogram equalization) juga merupakan teknik kompensasi fitur populer yang dipelajari dan dipraktekkan dengan sangat baik pada pemrosesan gambar untuk menormalkan fitur visual citra digital, misalnya kecerahan, skala tingkat abu-abu, kontras, dan lainnya. Secara umum, pemerataan histogram adalah cara untuk mengotomatisasi prosedur matematika yang memaksimalkan entropi citra. Transformasi pemerataan histogram dihitung menggunakan persamaan berikut:

$$S(m_i) = (m_0 + m_1 + \dots + m_i)(K - 1) / m \dots (2.5)$$

Di mana m adalah total jumlah piksel ($m = m_0 + m_1 + \dots + m_{K-1}$) dan m_j adalah jumlah piksel dalam gambar yang memiliki nilai tingkat abu-abu tertentu. Fungsi transformasi citra adalah fungsi distribusi kumulatif (cumulative distribution function/CDF) dari nilai piksel dalam citra yang dinormalisasi untuk memetakan rentang nilai piksel ke rentang yang diinginkan. Dalam implementasi metode Histogram Equalization pada pengolahan citra digital, persamaan matematikanya dapat dinotasikan sebagai berikut:

$$E(i,j) = (\text{GrayLevel} / \text{Area}) \times H_{ck} \dots (2.6)$$

Dengan $D(i,j)$ adalah matriks citra masukan, i dan j adalah indeks baris dan kolom pada matriks, N dan M adalah jumlah baris dan kolom pada matriks, H_{ck} adalah nilai kumulatif histogram, GrayLevel adalah tingkat keabuan (biasanya bernilai 256), Area adalah luas matriks citra masukan ($N \times M$), dan $E(i,j)$ adalah citra keluaran (hasil). Nilai kumulatif histogram sendiri didefinisikan sebagai $H_{ck} = \sum H_k$ dengan $\text{GrayLevel} = 256$.

Analisis histogram membandingkan histogram citra asli dengan histogram citra hasil proses suatu metode tertentu. Pemrosesan citra warna dari saluran RGB menjadi citra skala abu-abu mengakibatkan perubahan pada frekuensi histogram, di mana histogram citra mengalami penurunan sebesar $1/3$ dari histogram gambar warna dengan bins = 256 dan range 0 sampai 256.

2.5 Thresholding dan Segmentasi Warna

Segmentasi warna merupakan salah satu teknik penting dalam pengolahan citra digital yang bertujuan untuk memisahkan objek dari latar belakang berdasarkan warna atau intensitas piksel. Salah satu metode segmentasi yang paling dasar adalah thresholding, yaitu proses penentuan nilai ambang batas untuk mengelompokkan piksel ke dalam dua kelompok atau lebih. Dalam proses ini, piksel dengan nilai intensitas di atas threshold akan diubah menjadi putih (255), sedangkan piksel dengan nilai di bawah threshold akan diubah menjadi hitam (0). Metode thresholding banyak digunakan karena sederhana namun efektif untuk mendeteksi objek pada citra digital, terutama pada citra grayscale.

Dalam pengolahan citra digital sering muncul derau (noise), yaitu gangguan berupa bintik hitam atau putih yang dapat menurunkan kualitas gambar. Noise dapat disebabkan oleh gangguan optik saat pengambilan gambar maupun kesalahan proses pengolahan citra. Salah satu contoh noise yang umum adalah salt & pepper noise yang muncul secara acak pada citra. Oleh karena itu, proses segmentasi biasanya diikuti dengan teknik pengurangan noise agar hasil citra menjadi lebih bersih dan mudah dianalisis. Histogram citra juga memiliki peranan penting dalam proses thresholding karena histogram menunjukkan distribusi intensitas piksel dari nilai 0 hingga 255. Dengan melihat puncak dan lembah histogram, nilai threshold yang tepat dapat ditentukan sehingga objek dapat dipisahkan dengan lebih jelas dari background.

Selain thresholding, terdapat metode pemrosesan warna semu (pseudo-color) yang digunakan untuk mengubah citra grayscale menjadi citra berwarna berdasarkan fungsi pemetaan tertentu. Teknik ini memanfaatkan transformasi warna merah, hijau, dan biru untuk menghasilkan kombinasi warna yang berbeda sesuai tingkat intensitas grayscale. Piksel dengan intensitas rendah dapat ditampilkan sebagai warna gelap, sedangkan piksel dengan intensitas tinggi dapat ditampilkan sebagai warna yang lebih terang. Teknik ini membantu proses segmentasi dan analisis objek karena perbedaan intensitas dapat terlihat lebih jelas secara visual.

Untuk meningkatkan kualitas citra, digunakan juga metode Histogram Equalization yang berfungsi memperbaiki kontras gambar grayscale dengan membuat distribusi intensitas menjadi lebih merata. Metode ini mampu meningkatkan detail pada area gelap maupun terang sehingga citra menjadi lebih jelas. Selain itu, penggunaan Low-Pass Filter dapat membantu mengurangi noise sebelum dilakukan peningkatan kontras. Kombinasi kedua metode tersebut menghasilkan kualitas citra yang lebih baik dan nyaman untuk diamati.

Perkembangan teknologi pengolahan citra digital saat ini memungkinkan berbagai teknik seperti konversi RGB ke grayscale, thresholding, histogram, brightness, contrast, dan segmentasi warna diterapkan menggunakan library OpenCV pada Python. Implementasi metode tersebut terbukti mampu menangani citra sederhana maupun kompleks dengan baik sehingga gambar yang semula gelap, kurang jelas, atau memiliki noise dapat diperbaiki dan dianalisis dengan lebih mudah.

BAB III

HASIL

Pada bagian ini akan ditampilkan hasil dari praktikum UTS yang telah dikerjakan, meliputi screenshot hasil pengolahan citra, histogram, serta output lainnya yang diperoleh selama proses praktikum. Hasil tersebut digunakan untuk menunjukkan proses kerja program sekaligus menganalisis perubahan atau pengaruh yang terjadi pada citra setelah dilakukan pengolahan.

3.1 Ekstraksi Kanal Warna, Ambang Batas dan Analisis Histogram



Gambar diatas merupakan sebuah citra yang bertuliskan nama lengkap yang ditulis tangan menggunakan spidol dengan tinta merah, hijau, dan biru di atas kertas putih. Urutan warna tersebut di mulai dari **Tashfia** diberi warna **hijau**, **Wahdah** diberi warna **biru**, dan **Fuady** diberi warna **merah**.

Berikut program yang dikerjakan:

Import Library

```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Kode program di atas digunakan untuk membaca dan mengolah gambar menggunakan Python dengan bantuan library OpenCV, NumPy, dan Matplotlib. Pada bagian awal, dilakukan **import library** terlebih dahulu. **cv2** digunakan untuk pengolahan citra atau gambar, **numpy** digunakan untuk pengolahan data berbentuk array atau matriks, sedangkan **matplotlib.pyplot** digunakan untuk menampilkan gambar maupun grafik hasil pengolahan.

Membaca file gambar

```
In [2]: img = cv2.imread("namaku.jpg")
```

Selanjutnya, program membaca file gambar bernama **"namaku.jpg"** menggunakan fungsi **cv2.imread()**. Gambar yang dibaca kemudian disimpan ke dalam variabel **img**.

Melihat dimensi gambar

```
In [3]: img.shape
```

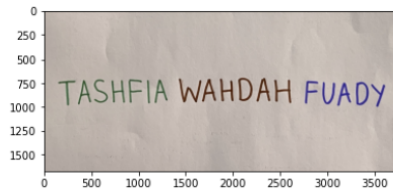
```
Out[3]: (1672, 3727, 3)
```

Setelah gambar berhasil dibaca, program menggunakan `img.shape` untuk melihat dimensi gambar. Hasilnya berupa ukuran tinggi 720, lebar 1280, dan jumlah channel warna pada gambar.

Menampilkan gambar asli menggunakan plt

```
In [4]: plt.imshow(img)
```

```
Out[4]: <matplotlib.image.AxesImage at 0x16d1177da60>
```

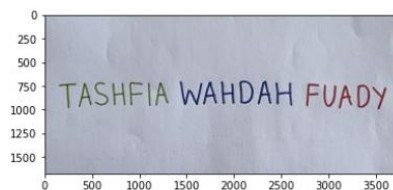


Berikutnya, gambar asli ditampilkan menggunakan `plt.imshow(img)`. Namun, karena OpenCV membaca gambar dalam format warna BGR (Blue, Green, Red), warna gambar yang tampil terlihat tidak sesuai dengan warna aslinya.

Konversi gambar asli ke rgb dan menampilkannya

```
In [5]: rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        plt.imshow(rgb)
```

```
Out[5]: <matplotlib.image.AxesImage at 0x16d1186fc40>
```



Oleh karena itu, dilakukan konversi warna dari BGR menjadi RGB menggunakan fungsi `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`. Hasil konversi disimpan ke dalam variabel `rgb` agar gambar dapat ditampilkan dengan warna yang benar saat menggunakan Matplotlib.

Deklarasi variabel baru untuk setiap channel warna

```
In [6]: r = rgb[:, :, 0]
        g = rgb[:, :, 1]
        b = rgb[:, :, 2]
```

Pada bagian ini, program memisahkan gambar RGB menjadi tiga channel warna, yaitu merah (red), hijau (green), dan biru (blue). Variabel `r` mengambil channel merah, `g` mengambil channel hijau, dan `b` mengambil channel biru dari gambar RGB. Pemisahan channel warna ini berguna untuk menganalisis atau mengolah setiap warna secara terpisah dalam proses pengolahan citra digital.

a. Deteksi Warna

Menampilkan hasil setiap warna

```
In [7]: fig, axs = plt.subplots(2,2, figsize=(15,8))

axs[0,0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
axs[0,0].set_title('Gambar Asli')

axs[0,1].imshow(r, cmap="gray")
axs[0,1].set_title('Deteksi Merah')

axs[1,0].imshow(g, cmap="gray")
axs[1,0].set_title('Deteksi Hijau')

axs[1,1].imshow(b, cmap="gray")
axs[1,1].set_title('Deteksi Biru')

Out[7]: Text(0.5, 1.0, 'Deteksi Biru')
```

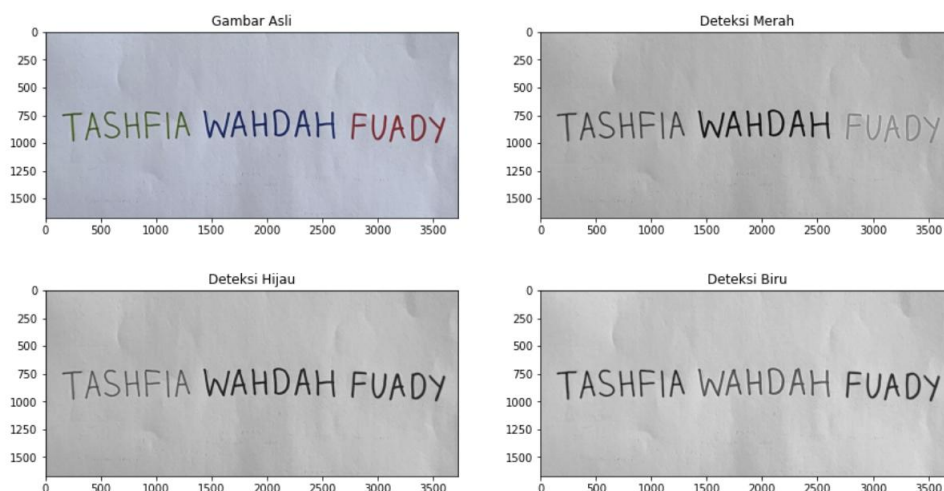
Kode program di atas digunakan untuk menampilkan gambar asli serta hasil pemisahan channel warna merah, hijau, dan biru dalam satu tampilan menggunakan library Matplotlib. Dengan tampilan ini, proses pengolahan citra menjadi lebih mudah diamati dan dibandingkan.

Pada bagian awal, digunakan perintah `fig, axs = plt.subplots(2,2, figsize=(15,8))` untuk membuat area tampilan dengan 2 baris dan 2 kolom sehingga tersedia empat subplot. Parameter `figsize` digunakan untuk mengatur ukuran tampilan agar gambar terlihat lebih jelas.

Selanjutnya, program menampilkan gambar asli pada subplot pertama. Sebelum ditampilkan, gambar dikonversi dari format BGR ke RGB menggunakan `cv2.cvtColor()` agar warna tampil sesuai aslinya. Setelah itu, subplot diberi judul “Gambar Asli”.

Berikutnya, program menampilkan channel merah, hijau, dan biru secara terpisah menggunakan mode grayscale (gray). Variabel `r`, `g`, dan `b` masing-masing menyimpan intensitas warna merah, hijau, dan biru dari gambar RGB. Pada tampilan grayscale, area yang lebih terang menunjukkan intensitas warna yang lebih tinggi pada channel tersebut. Dengan cara ini, pengguna dapat melihat bagian gambar mana yang dominan merah, hijau, atau biru dengan lebih jelas.

Berikut output citra yang dihasilkan:



Berdasarkan analisis pada pengolahan citra, proses ini bertujuan untuk melihat "kadar" warna dasar (Merah, Hijau, dan Biru) yang membentuk setiap kata pada gambar tersebut.

Deteksi Merah: Kata "FUADY" terlihat paling samar (paling terang/abu-abu muda) karena memiliki nilai intensitas merah yang tinggi, sementara "WAHDAH" yang berwarna biru terlihat sangat gelap (hitam) karena tidak memiliki unsur merah sama sekali.

Deteksi Hijau: Kata "TASHFIA" tampak lebih terang dibandingkan kata "WAHDAH", yang menandakan channel hijau berhasil menangkap pigmen warna hijau pada tulisan tersebut. Namun, latar belakang kertas masih terlihat sangat putih karena kertas putih secara alami mengandung nilai hijau yang tinggi.

Deteksi Biru: Kata "WAHDAH" adalah yang paling terang di sini karena warna aslinya adalah biru murni. Kata "TASHFIA" dan "FUADY" terlihat lebih gelap, menunjukkan minimnya unsur biru pada kedua kata tersebut.

Hal yang sama terjadi pada panel lainnya, kata akan terlihat terang di channel yang sesuai dengan warna aslinya dan akan meredup atau menghitam jika warna aslinya berbeda. Semakin terang atau putih suatu objek pada hasil deteksi channel warna tertentu, menandakan bahwa objek tersebut memiliki unsur warna dasar tersebut yang sangat kuat. Melalui cara ini, bisa memisahkan setiap teks berdasarkan warnanya hanya dengan melihat seberapa terang tulisan tersebut muncul di setiap channel.

b. Histogram

Histogram

```
In [10]: # Membuat area subplot 4 baris dan 2 kolom
fig, axs = plt.subplots(4, 2, figsize=(15, 20))

# Menghitung histogram gambar asli untuk channel RGB
hist_asli_r = cv2.calcHist([img], [0], None, [256], [0, 256])
hist_asli_g = cv2.calcHist([img], [1], None, [256], [0, 256])
hist_asli_b = cv2.calcHist([img], [2], None, [256], [0, 256])

# Menampilkan gambar asli
axs[0, 0].imshow(img)
axs[0, 0].set_title('Gambar Asli')

# Menampilkan histogram RGB gambar asli
axs[0, 1].plot(hist_asli_r, color='r')
axs[0, 1].plot(hist_asli_g, color='g')
axs[0, 1].plot(hist_asli_b, color='b')
axs[0, 1].set_title('Histogram Asli (RGB)')
```

Kode program di atas digunakan untuk menampilkan histogram warna pada gambar serta melihat distribusi intensitas setiap channel RGB, yaitu merah, hijau, dan biru. Histogram berfungsi untuk menunjukkan jumlah piksel pada setiap tingkat intensitas warna dari 0–255 sehingga warna yang dominan pada gambar dapat diketahui.

Pada bagian awal, program membuat area tampilan menggunakan `plt.subplots(4, 2, figsize=(15, 20))` untuk menampilkan gambar dan histogram dalam 4 baris dan 2 kolom. Ukuran tampilan diperbesar agar hasil lebih jelas dan mudah dianalisis.

Selanjutnya, program menghitung histogram gambar asli menggunakan fungsi `cv2.calcHist()`. Histogram dihitung untuk channel merah, hijau, dan biru, lalu hasilnya disimpan ke dalam beberapa variabel. Setelah itu, gambar asli ditampilkan agar dapat dibandingkan dengan histogramnya.

Program kemudian menampilkan histogram RGB dalam bentuk grafik berwarna merah, hijau, dan biru. Grafik ini menunjukkan persebaran intensitas warna pada gambar. Jika grafik suatu warna lebih tinggi, maka warna tersebut lebih dominan.

Pada bagian berikutnya, program menghitung dan menampilkan histogram masing-masing channel warna secara terpisah beserta gambar grayscale-nya. Tampilan ini memudahkan pengguna untuk melihat distribusi intensitas warna merah, hijau, dan biru pada gambar secara lebih detail.

```

# Merah
hist_r = cv2.calcHist([r], [0], None, [256], [0, 256])
axs[1, 0].imshow(r, cmap='gray')
axs[1, 0].set_title('Channel Merah')
axs[1, 1].plot(hist_r, color='r')
axs[1, 1].set_title('Histogram Merah')

# Hijau
hist_g = cv2.calcHist([g], [0], None, [256], [0, 256])
axs[2, 0].imshow(g, cmap='gray')
axs[2, 0].set_title('Channel Hijau')
axs[2, 1].plot(hist_g, color='g')
axs[2, 1].set_title('Histogram Hijau')

# Biru
hist_b = cv2.calcHist([b], [0], None, [256], [0, 256])
axs[3, 0].imshow(b, cmap='gray')
axs[3, 0].set_title('Channel Biru')
axs[3, 1].plot(hist_b, color='b')
axs[3, 1].set_title('Histogram Biru')

plt.tight_layout()
plt.show()

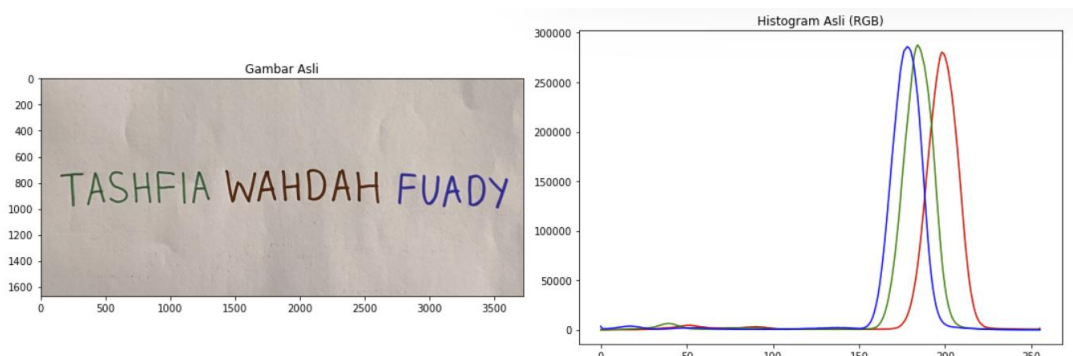
```

Untuk channel merah, program menghitung histogram. Channel merah kemudian ditampilkan dalam bentuk grayscale menggunakan `imshow(r, cmap='gray')`. Tampilan grayscale digunakan agar intensitas warna lebih mudah diamati. Area yang terang menunjukkan bagian gambar yang memiliki kandungan warna merah tinggi. Histogram merah kemudian ditampilkan dengan garis berwarna merah. Proses yang sama dilakukan pada channel hijau dan biru. Channel hijau ditampilkan dalam grayscale dan histogramnya digambar menggunakan warna hijau. Semakin terang area gambar, maka semakin besar intensitas warna hijau pada bagian tersebut. Begitu juga dengan channel biru yang menampilkan intensitas warna biru pada gambar beserta histogramnya.

Pada bagian akhir, program menggunakan `plt.tight_layout()` untuk mengatur jarak antar subplot agar tampilan lebih rapi dan tidak saling bertumpuk, sedangkan `plt.show()` digunakan untuk menampilkan seluruh hasil visualisasi.

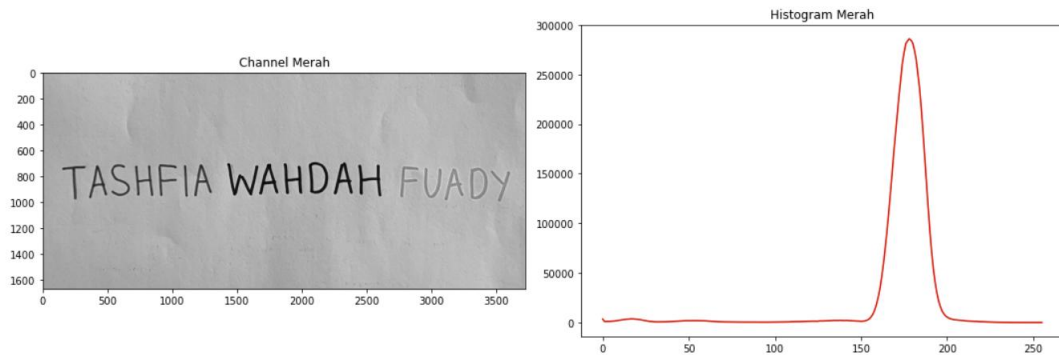
Berikut output histogram yang dihasilkan:

Berdasarkan output histogram, program berhasil menampilkan distribusi intensitas warna RGB pada gambar asli serta masing-masing channel merah, hijau, dan biru.

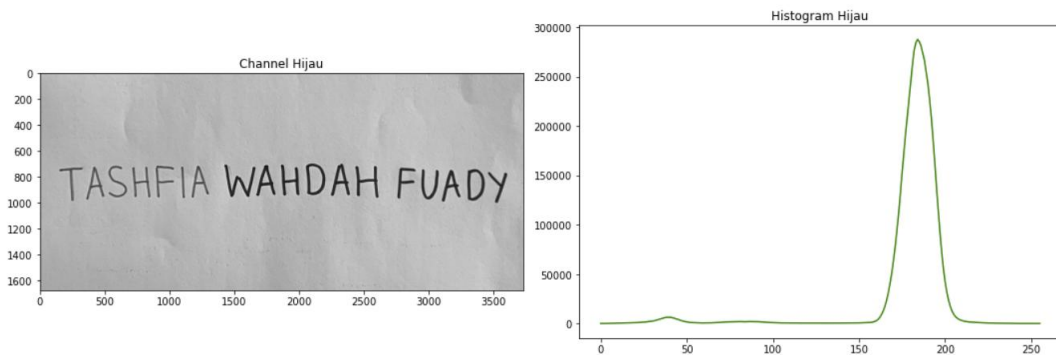


Pada gambar asli terlihat tulisan “TASHFIA WAHDAH FUADY” dengan warna hijau, merah, dan biru di atas latar belakang putih terang. Histogram RGB menunjukkan tiga grafik warna yang saling berdekatan karena latar belakang putih memengaruhi seluruh channel warna secara hampir sama. Pada intensitas 0–100 terlihat gelombang kecil yang mewakili piksel tulisan, sedangkan pada rentang 100–150 terdapat celah yang menunjukkan pemisahan antara objek teks dan latar belakang sehingga cocok digunakan untuk thresholding. Puncak besar pada intensitas sekitar

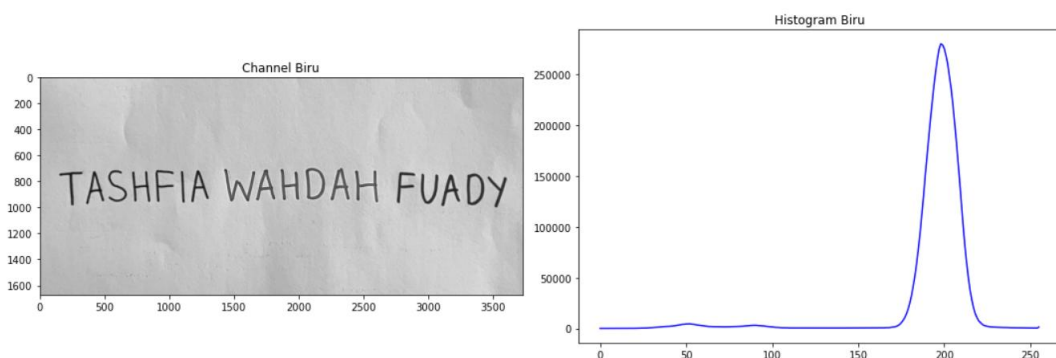
175–225 menunjukkan dominasi piksel putih dari latar belakang, dan bagian atas grafik tampak terpotong agar distribusi intensitas tulisan tetap terlihat jelas.



Pada channel merah, histogram merah menunjukkan puncak cukup tinggi pada intensitas menengah hingga terang, yang berarti banyak piksel merah berada pada rentang tersebut.



Pada channel hijau, histogram hijau memiliki bentuk yang hampir sama dengan histogram merah, tetapi terdapat sedikit perbedaan tinggi grafik karena distribusi warna hijau pada gambar berbeda dengan warna merah.



Pada channel biru, histogram biru memiliki puncak yang cukup tinggi, menandakan bahwa intensitas warna biru cukup banyak pada gambar, terutama pada area tulisan tersebut.

Secara keseluruhan, histogram ketiga channel memiliki pola yang hampir sama karena dipengaruhi oleh latar belakang putih yang terdapat pada seluruh gambar. Namun, perbedaan tinggi puncak histogram menunjukkan dominasi warna tertentu pada masing-masing tulisan. Dari hasil ini dapat disimpulkan bahwa proses pemisahan channel warna dan analisis histogram berhasil dilakukan dengan baik sehingga distribusi warna pada gambar dapat dianalisis dengan jelas.

c. Nilai Ambang Batas

Nilai Ambang Batas

```
In [9]: # Memisahkan channel warna menjadi tiga
b, g, r = cv2.split(img)

fig, axs = plt.subplots(2, 2, figsize=(12, 10))

# Mengisolasi bagian merah
selisih_merah = cv2.subtract(r, g)
_, hasil_merah = cv2.threshold(selisih_merah, 40, 255, cv2.THRESH_BINARY)

# Mengisolasi bagian biru
selisih_biru = cv2.subtract(b, r)
_, hasil_biru = cv2.threshold(selisih_biru, 40, 255, cv2.THRESH_BINARY)

# Mengisolasi bagian hijau
selisih_hijau = cv2.subtract(g, r)
_, hasil_hijau = cv2.threshold(selisih_hijau, 18, 255, cv2.THRESH_BINARY)
```

Kode program di atas digunakan untuk melakukan proses thresholding atau nilai ambang batas berdasarkan channel warna merah, hijau, dan biru. Pada awal program, gambar dipisahkan menjadi tiga channel warna menggunakan `cv2.split(img)` sehingga diperoleh channel biru (b), hijau (g), dan merah (r). Setelah itu, program membuat subplot 2×2 untuk menampilkan hasil proses.

Selanjutnya, program mengisolasi warna merah dengan menghitung selisih antara channel merah dan hijau menggunakan `cv2.subtract(r, g)`. Jika nilai merah lebih besar, maka area tersebut dianggap dominan merah. Hasil selisih kemudian diproses menggunakan `cv2.threshold()` dengan nilai ambang batas 40. Piksel yang nilainya lebih besar dari 40 akan berubah menjadi putih, sedangkan sisanya menjadi hitam sehingga objek merah terlihat lebih jelas.

Proses yang sama dilakukan pada warna biru dengan menghitung selisih antara channel biru dan merah menggunakan threshold 40. Sedangkan untuk warna hijau digunakan selisih antara channel hijau dan merah dengan threshold 18. Perbedaan nilai threshold ini digunakan agar proses deteksi setiap warna menjadi lebih optimal sesuai intensitas warna pada gambar.

```
# NONE (Hitam)
hitam_polos = np.zeros_like(r)
axs[0,0].imshow(hitam_polos, cmap='gray')
axs[0,0].set_title('NONE')

# BLUE
axs[0,1].imshow(hasil_merah, cmap='gray')
axs[0,1].set_title('BLUE')

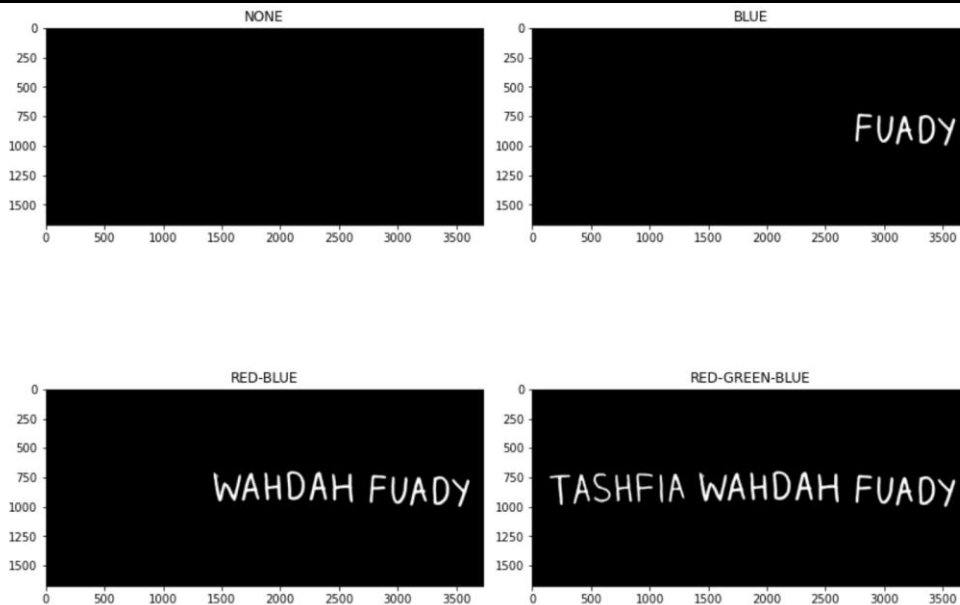
# RED-BLUE
gabung_merah_biru = cv2.bitwise_or(hasil_merah, hasil_biru)
axs[1,0].imshow(gabung_merah_biru, cmap='gray')
axs[1,0].set_title('RED-BLUE')

# RED-GREEN-BLUE
semua_warna = cv2.bitwise_or(gabung_merah_biru, hasil_hijau)
axs[1,1].imshow(semua_warna, cmap='gray')
axs[1,1].set_title('RED-GREEN-BLUE')

plt.tight_layout()
plt.show()
```

Pada bagian tampilan, subplot pertama menampilkan gambar hitam polos (NONE) menggunakan array kosong dari channel merah. Subplot kedua menampilkan hasil deteksi warna biru. Kemudian subplot ketiga menampilkan gabungan hasil deteksi merah dan biru menggunakan `cv2.bitwise_or()`, sehingga objek merah dan biru muncul bersamaan. Terakhir, subplot keempat menampilkan gabungan seluruh warna merah, hijau, dan biru sehingga semua tulisan dapat terlihat.

Berikut output yang dihasilkan:

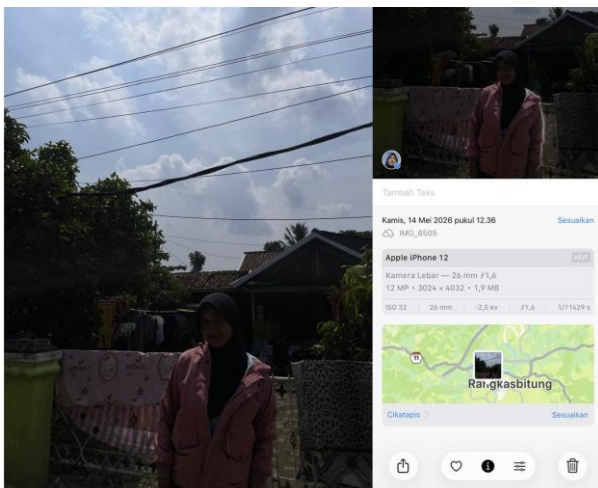


Berdasarkan output gambar yang dihasilkan, terlihat bahwa program berhasil memisahkan warna pada tulisan nama menjadi beberapa channel warna. Pada tampilan “BLUE”, hanya tulisan “FUADY” yang terlihat jelas karena tulisan tersebut memiliki dominasi warna biru. Pada tampilan “RED-BLUE”, tulisan “WAHDAH” dan “FUADY” terlihat karena merupakan gabungan channel merah dan biru. Sedangkan pada tampilan “RED-GREEN-BLUE”, seluruh tulisan yaitu “TASHFIA WAHDAH FUADY” terlihat jelas karena semua channel warna digabungkan kembali.

Dari hasil tersebut dapat dianalisis bahwa setiap channel warna hanya menampilkan objek yang memiliki intensitas warna dominan sesuai channel-nya. Tulisan “TASHFIA” muncul dominan pada channel hijau, “WAHDAH” pada channel merah, dan “FUADY” pada channel biru. Hal ini menunjukkan bahwa proses pemisahan channel warna berhasil dilakukan dengan baik dan setiap warna dapat diidentifikasi secara terpisah menggunakan teknik pengolahan citra digital.

3.2 Operasi Pixel

Dilakukan teknik pengolahan citra digital dalam memperbaiki gambar yang terkena efek backlight, yaitu kondisi ketika sumber cahaya berada di belakang objek sehingga wajah atau tubuh terlihat gelap. Pada praktikum ini dilakukan beberapa proses pengolahan citra seperti konversi gambar ke grayscale, peningkatan kecerahan, dan kontras untuk memperjelas objek utama serta mengurangi dominasi cahaya pada latar belakang.

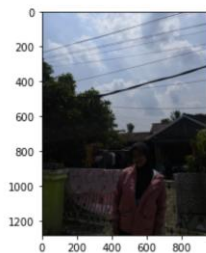


Gambar diatas merupakan gambar yang diambil menggunakan perangkat Iphone 12 pada pukul 12.36 di siang hari yang menampilkan suasana perumahan dilengkapi dengan pepohonan, latar awan yang cerah, serta objek manusia (saya) yang berdiri membelakangi cahaya. Sehingga gambar yang dihasilkan terkena efek backlight.

Operasi Pixel

```
In [10]: image = cv2.imread("foto.jpg")
        rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        plt.imshow(rgb_image)

Out[10]: <matplotlib.image.AxesImage at 0x1ff888dcb80>
```

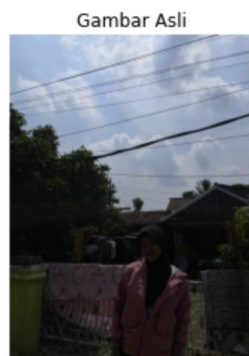


Kode program di atas digunakan untuk membaca dan menampilkan gambar menggunakan library OpenCV dan Matplotlib. Pada bagian awal, gambar dibaca dari file bernama “foto.jpg” menggunakan fungsi `cv2.imread()` dan disimpan ke dalam variabel `image`. Secara default, OpenCV membaca gambar dalam format warna BGR (Blue, Green, Red). Selanjutnya, program mengubah format warna gambar dari BGR menjadi RGB menggunakan `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`. Proses ini dilakukan agar warna gambar tampil sesuai dengan warna aslinya saat ditampilkan menggunakan Matplotlib. Hasil konversi kemudian disimpan ke dalam variabel `rgb_image`.

a. Menampilkan Gambar Asli

Menampilkan Gambar Asli

```
In [11]: plt.imshow(rgb_image)
        plt.title("Gambar Asli")
        plt.axis("off")
        plt.show()
```



Setelah itu, gambar ditampilkan menggunakan `plt.imshow(rgb_image)`. Perintah ini digunakan untuk menampilkan gambar pada area plot Matplotlib. Tulisan `<matplotlib.image.AxesImage ...>` yang muncul merupakan informasi objek gambar yang dihasilkan oleh Matplotlib dan bukan bagian dari gambar itu sendiri. Pada bagian “Menampilkan Gambar Asli”, program kembali menampilkan gambar RGB menggunakan `plt.imshow(rgb_image)`. Kemudian, `plt.title("Gambar Asli")` digunakan untuk memberikan judul pada gambar, sedangkan `plt.axis("off")` berfungsi

untuk menyembunyikan garis sumbu dan angka koordinat agar tampilan gambar terlihat lebih rapi. Terakhir, `plt.show()` digunakan untuk menampilkan hasil gambar secara keseluruhan.

b. Menampilkan Gambar Gray

Menampilkan Gambar Gray

```
In [26]: gambar_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
plt.imshow(gambar_gray, cmap = 'gray')
plt.title("Gambar Gray")
plt.axis("off")
plt.show()
```

Kode program di atas digunakan untuk mengubah gambar berwarna menjadi gambar grayscale atau abu-abu menggunakan fungsi `cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`. Proses grayscale membuat gambar hanya memiliki tingkat intensitas cahaya tanpa warna RGB, sehingga setiap piksel hanya menyimpan nilai kecerahan dari gelap ke terang. Hasil konversi disimpan ke dalam variabel `gambar_gray`. Setelah itu, gambar ditampilkan menggunakan `plt.imshow(gambar_gray, cmap='gray')` agar tampil dalam warna abu-abu. Fungsi `plt.title()` digunakan untuk memberi judul gambar, sedangkan `plt.axis("off")` digunakan untuk menyembunyikan garis sumbu agar tampilan lebih rapi.

Gambar Gray



Berdasarkan output yang dihasilkan, gambar grayscale terlihat cukup gelap terutama pada bagian objek manusia di depan rumah. Hal ini terjadi karena adanya efek backlight dari cahaya langit yang sangat terang di bagian belakang gambar. Area langit tampak memiliki tingkat kecerahan paling tinggi sehingga terlihat putih terang, sedangkan area tubuh, wajah, dan pakaian objek terlihat lebih gelap. Detail pada wajah belum terlalu jelas karena intensitas cahaya pada objek kalah dibanding latar belakang. Selain itu, bagian pepohonan dan rumah terlihat berwarna abu-abu gelap sehingga teksturnya masih kurang terlihat dengan jelas.

c. Membuat Gambar Gray yang Dicerahkan

Membuat Gambar Gray yang di Cerahkan

```
In [28]: [baris, kolom] = gambar_gray.shape[:2]

beta = 18
gambar_cerah = np.zeros((baris, kolom, 3))

for x in range(baris):
    for y in range(kolom):
        gyx = gambar_gray[x,y] + beta
        gambar_cerah[x,y] = gyx

gambar_cerah = gambar_cerah.astype(np.uint8)

plt.imshow(gambar_cerah, cmap='gray')
plt.title("Gambar Gray yang Dipercahkan")
plt.axis("off")
plt.show()
```

Kode program ini digunakan untuk meningkatkan kecerahan gambar grayscale. Program mengambil ukuran gambar menggunakan `gambar_gray.shape[:2]` lalu menyimpannya ke variabel `baris` dan `kolom`. Variabel `beta = 18` digunakan sebagai nilai brightness yang akan ditambahkan ke setiap piksel gambar. Program membuat array kosong menggunakan `np.zeros()` untuk menyimpan hasil gambar yang telah diproses. Selanjutnya, dilakukan perulangan pada setiap piksel gambar menggunakan `for`. Nilai setiap piksel grayscale ditambah dengan nilai `beta` menggunakan rumus `gyx = gambar_gray[x,y] + beta`. Hasil penjumlahan disimpan ke dalam `gambar_cerah`. Setelah semua piksel diproses, gambar diubah ke tipe data `uint8` menggunakan `astype(np.uint8)` agar dapat ditampilkan dengan benar.

Gambar Gray yang Dipercahkan



Berdasarkan outputnya, tingkat kecerahan gambar meningkat dibanding grayscale asli. Area wajah dan pakaian objek mulai terlihat lebih terang sehingga bentuk tubuh lebih mudah dikenali. Bagian rumah dan pepohonan juga tampak sedikit lebih jelas karena intensitas piksel bertambah. Langit pada latar belakang menjadi semakin terang mendekati putih, sedangkan bayangan gelap pada objek mulai berkurang. Meskipun demikian, detail-detail kecil seperti tekstur wajah dan

pakaian masih belum terlalu tajam karena proses ini hanya menambah brightness tanpa meningkatkan kontras gambar.

d. Membuat Gambar Gray yang Dikontraskan

Membuat Gambar Gray yang di Kontraskan

```
In [30]: alpha = 1.05
gambar_kontras = np.zeros((baris, kolom, 3))

for x in range(baris):
    for y in range(kolom):
        gyx = gambar_gray[x,y] * alpha
        gambar_kontras[x,y] = gyx

gambar_kontras = gambar_kontras.astype(np.uint8)

plt.imshow(gambar_kontras, cmap='gray')
plt.title("Gambar Gray yang Diperkontras")
plt.axis("off")
plt.show()
```

Kode program ini digunakan untuk meningkatkan kontras gambar grayscale. Variabel $\alpha = 1.05$ digunakan sebagai faktor pengali kontras. Nilai α yang lebih besar dari 1 akan membuat perbedaan antara area terang dan gelap menjadi lebih jelas. Program membuat array kosong bernama `gambar_kontras` untuk menyimpan hasil proses. Setelah itu, dilakukan perulangan pada setiap piksel gambar. Nilai piksel grayscale dikalikan dengan α menggunakan rumus $gyx = gambar_gray[x,y] * \alpha$. Hasil proses kemudian disimpan ke dalam `gambar_kontras` dan diubah ke tipe data `uint8` agar dapat ditampilkan dengan baik.

Gambar Gray yang Diperkontras



Berdasarkan output yang dihasilkan, gambar terlihat memiliki perbedaan terang dan gelap yang lebih tegas. Area langit tampak lebih terang, sedangkan bagian pohon, rumah, dan objek manusia menjadi lebih gelap dibanding sebelumnya. Efek ini membuat garis tepi objek terlihat lebih jelas sehingga bentuk rumah, kabel listrik, dan pepohonan lebih mudah dibedakan dari latar belakang. Pada bagian tubuh dan pakaian objek, detail lipatan dan bentuk pakaian mulai lebih terlihat.

Namun, karena brightness tidak ditambah, area wajah masih terlihat cukup gelap akibat efek backlight yang kuat.

e. Membuat Gambar Gray yang Dipercerah dan Diperkontras

Membuat Gambar Gray yang dipercerah dan diperkontras

```
In [32]: beta = 18
alpha = 1.0
gambar_hasil = np.zeros((baris, kolom, 3))

for x in range(baris):
    for y in range(kolom):
        gyx = alpha * gambar_gray[x,y] + beta
        gambar_hasil[x,y] = gyx

gambar_hasil = gambar_hasil.astype(np.uint8)

plt.imshow(gambar_hasil, cmap='gray')
plt.title("Gambar Gray Dipercerah dan Diperkontras")
plt.axis("off")
plt.show()
```

Kode program ini digunakan untuk menggabungkan proses peningkatan brightness dan contrast sekaligus. Variabel $\beta = 18$ digunakan untuk menambah kecerahan, sedangkan $\alpha = 1.0$ digunakan untuk mengatur kontras gambar. Program membuat array kosong bernama `gambar_hasil` untuk menyimpan hasil akhir pengolahan citra. Selanjutnya, program melakukan perulangan pada setiap piksel gambar grayscale. Nilai piksel diproses menggunakan rumus $gyx = \alpha * gambar_gray[x,y] + \beta$. Rumus tersebut akan mengalikan nilai piksel dengan α lalu menambahkan β sehingga gambar menjadi lebih terang dan detail objek lebih jelas. Hasil proses disimpan ke dalam `gambar_hasil`, kemudian diubah menjadi tipe data `uint8`.

Gambar Gray Dipercerah dan Diperkontras



Berdasarkan output yang dihasilkan, gambar terlihat paling jelas dibanding proses sebelumnya. Tingkat kecerahan meningkat sehingga wajah, pakaian, dan tubuh objek lebih mudah terlihat. Area rumah, pepohonan, dan kabel listrik juga tampak lebih jelas karena detail objek menjadi lebih tegas. Langit masih terlihat terang, tetapi objek di bagian depan tidak terlalu gelap lagi sehingga efek backlight berhasil dikurangi. Kombinasi brightness dan contrast membuat gambar

terlihat lebih seimbang antara area terang dan gelap sehingga objek utama dapat diamati dengan lebih baik.

f. Menampilkan Hasil Semua Gambar

Menampilkan Hasil Semua Gambar

```
In [33]: fig, axs = plt.subplots(5, figsize=(10,20))
axs[0].imshow(rgb_image)
axs[0].set_title('Gambar Asli')

axs[1].imshow(gambar_gray, cmap='gray')
axs[1].set_title('Gambar Gray')

axs[2].imshow(gambar_cerah, cmap='gray')
axs[2].set_title('Gambar Gray yang Dipercerah')

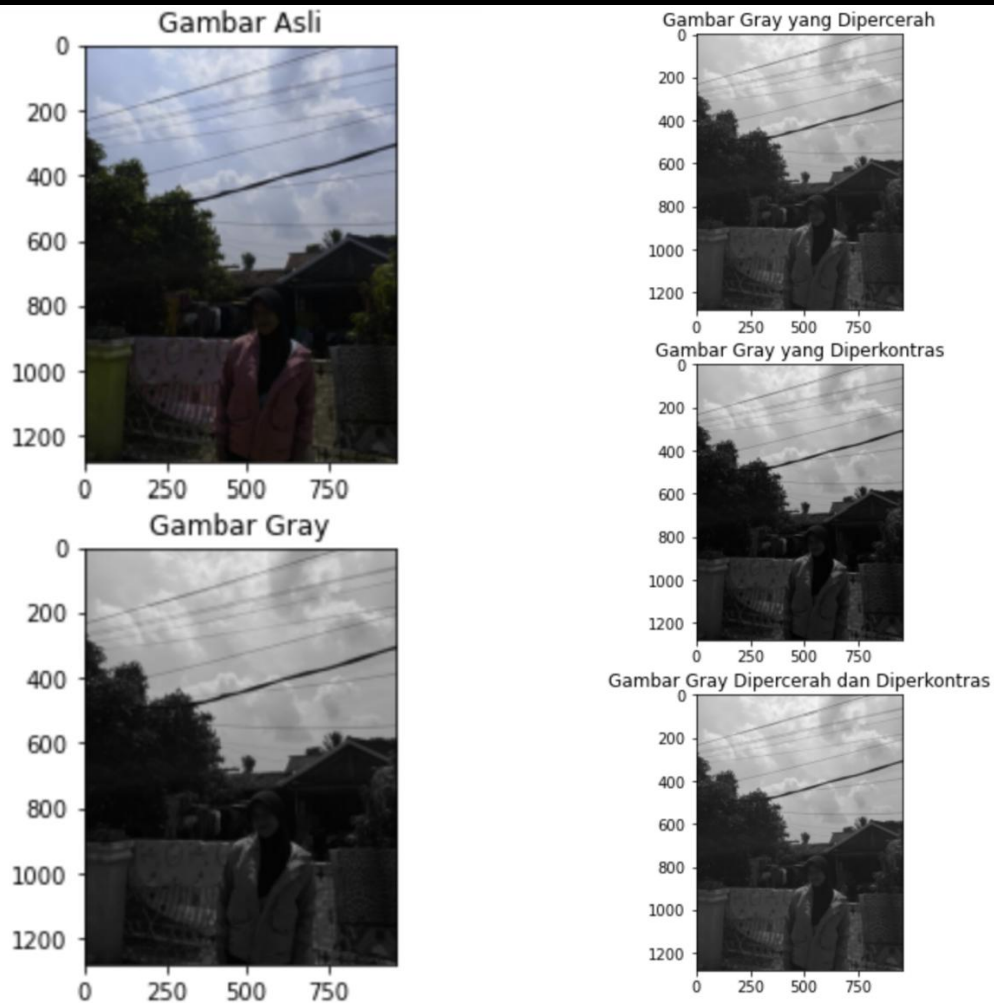
axs[3].imshow(gambar_kontras, cmap='gray')
axs[3].set_title('Gambar Gray yang Diperkontras')

axs[4].imshow(gambar_hasil, cmap='gray')
axs[4].set_title('Gambar Gray Dipercerah dan Diperkontras')
```

Kode program di atas digunakan untuk menampilkan seluruh hasil pengolahan gambar dalam satu tampilan menggunakan library Matplotlib. Program membuat area subplot menggunakan `plt.subplots(5, figsize=(10,20))` sehingga tersedia lima tampilan gambar yang disusun secara vertikal. Variabel `axs` digunakan untuk mengatur posisi masing-masing gambar agar hasil pengolahan citra dapat dibandingkan dengan lebih mudah.

Pada bagian pertama, program menampilkan gambar asli berwarna sebagai pembanding sebelum dilakukan pengolahan citra. Selanjutnya, program menampilkan gambar grayscale yang merupakan hasil konversi gambar berwarna menjadi abu-abu sehingga hanya menampilkan tingkat intensitas cahaya. Setelah itu, gambar grayscale yang telah dicerahkan ditampilkan menggunakan `gambar_cerah`, sehingga gambar terlihat lebih terang karena nilai brightness ditambahkan pada setiap piksel.

Berikutnya, program menampilkan gambar grayscale yang telah diperkontras menggunakan `gambar_kontras`, sehingga perbedaan antara area terang dan gelap terlihat lebih jelas dan detail objek lebih tegas. Terakhir, program menampilkan gambar hasil gabungan brightness dan contrast menggunakan `gambar_hasil`. Hasil ini menjadi yang paling baik karena gambar terlihat lebih terang sekaligus lebih jelas, sehingga objek dan detail pada gambar lebih mudah diamati.



Berdasarkan output yang dihasilkan, gambar asli terlihat memiliki efek backlight sehingga objek manusia tampak lebih gelap dibanding langit di belakang saya. Pada gambar grayscale, warna gambar berubah menjadi abu-abu dan detail objek masih terlihat cukup gelap. Pada gambar yang dicerahkan, wajah, pakaian, dan area sekitar objek mulai terlihat lebih terang, namun detail masih belum terlalu tajam. Pada gambar yang diperkontras, batas antara area terang dan gelap terlihat lebih jelas sehingga bentuk rumah, pepohonan, dan tubuh objek lebih mudah dikenali. Sedangkan pada gambar grayscale yang dicerahkan dan diperkontras, objek terlihat paling jelas karena gambar menjadi lebih terang sekaligus lebih detail. Efek backlight berkurang sehingga wajah, tubuh, dan latar belakang dapat diamati dengan lebih baik.

BAB IV

PENUTUP

Berdasarkan landasan teori yang telah dipelajari, dapat disimpulkan bahwa pengolahan citra digital merupakan teknik untuk memperbaiki, menganalisis, dan memproses gambar agar informasi visual lebih mudah diamati dan diinterpretasikan. Proses pengolahan citra meliputi konversi RGB ke grayscale, operasi brightness dan contrast, analisis histogram, thresholding, serta segmentasi warna. Konversi RGB ke grayscale digunakan untuk menyederhanakan citra dengan mengubah tiga channel warna menjadi satu tingkat intensitas keabuan sehingga proses komputasi menjadi lebih efisien. Operasi brightness dan contrast berfungsi meningkatkan kualitas visual citra, terutama pada gambar yang terlalu gelap atau terkena efek backlight. Histogram digunakan untuk melihat distribusi intensitas piksel sehingga dapat membantu analisis kualitas citra dan menentukan nilai threshold yang tepat. Sementara itu, thresholding dan segmentasi warna memungkinkan pemisahan objek berdasarkan intensitas atau warna tertentu sehingga objek pada citra dapat dikenali dengan lebih jelas. Seluruh proses tersebut dapat diimplementasikan menggunakan library OpenCV pada Python untuk menghasilkan pengolahan citra yang lebih efektif dan efisien.

Berdasarkan hasil praktikum yang telah dilakukan, proses pengolahan citra digital berhasil diterapkan dengan baik menggunakan Python, OpenCV, NumPy, dan Matplotlib. Pada praktikum ekstraksi channel warna, setiap tulisan berhasil dipisahkan berdasarkan warna merah, hijau, dan biru sehingga distribusi warna dapat dianalisis dengan jelas melalui histogram dan proses thresholding. Histogram menunjukkan persebaran intensitas piksel pada setiap channel warna serta membantu melihat dominasi warna pada citra. Proses thresholding juga berhasil memisahkan objek berdasarkan warna tertentu sehingga tulisan dapat dikenali sesuai channel warnanya. Pada praktikum operasi piksel, proses grayscale, peningkatan brightness, contrast, serta kombinasi keduanya berhasil memperbaiki gambar yang terkena efek backlight. Hasil terbaik diperoleh pada kombinasi brightness dan contrast karena gambar menjadi lebih terang, detail objek lebih jelas, dan efek cahaya belakang dapat dikurangi. Secara keseluruhan, praktikum ini menunjukkan bahwa teknik pengolahan citra digital mampu meningkatkan kualitas citra serta membantu proses analisis visual menjadi lebih mudah dan jelas.

DAFTAR PUSTAKA

- Bastian, A., Zaliluddin, D., & Al Maroghi, M. S. (2022). Implementasi Pemrograman Paralel Menggunakan Platform OpenMP pada Citra Digital dengan Metode Low-Pass Filter dan Histogram Equalization. *INFOTECH Journal*, 8(1), 28–33. <https://doi.org/10.31949/infotech.v8i1.1878>
- Fajri, R. (2021). Pengolahan Citra Untuk Meningkatkan Kualitas Gambar Menggunakan Metode Kecerahan Citra Kontras Dan Penajaman Citra. *Lentera : Jurnal Ilmiah Sains, Teknologi, Ekonomi, Sosial, Dan Budaya*, 5(6). <https://journal.umuslim.ac.id/index.php/ltr2/article/view/899>
- Kusnadi, K., Riana, D., & Syahrani, M. (2022). PENGELOLAAN CITRA DIGITAL DENGAN MENGGUNAKAN METODE TRANSFORMASI GRAYSCALE DAN PEMERATAAN HISTOGRAM. *JTIK (Jurnal Teknik Informatika Kaputama)*, 6(1), 108–119.
- Suhada, I., Ermawati, A., Paris Rifandie Maulidhan, M., & Rosyani, P. (2024). Konversi Citra RGB Ke Grayscale Menggunakan OpenCV. *JRIIN : Jurnal Riset Informatika Dan Inovasi*, 2(3), 390–394. Retrieved from <https://jurnalmahasiswa.com/index.php/jriin/article/view/1436>